



UNIVERSIDAD
DE ANTIOQUIA

EQUIPO



**UNIVERSIDAD
DE ANTIOQUIA**

1 8 0 3

Presentado por:

*María Alejandra Otálvaro
Isabella Sánchez Mejía
Nicole Mariana Adarve*

**Facultad de Ingeniería
Universidad de Antioquia**

**Estructura de datos y Lab
2026**



UNIVERSIDAD
DE ANTIOQUIA

HASH BÁSICO Y HASHING ESTÁTICO

AGENDA



Hash básico

1

- ▶ Definición **1.1**
- ▶ ¿Cómo funciona? **1.2**
- ▶ Técnicas del hash y manejos de colisiones **1.3**
- ▶ Ventajas y desventajas **1.4**
- ▶ Aplicaciones **1.5**



Hash estático

2

- ▶ Definición **2.1**
- ▶ ¿Cómo funciona? **2.2**
- ▶ Técnicas del hash y manejos de colisiones **2.3**
- ▶ Ventajas y desventajas **2.4**
- ▶ Aplicaciones **2.5**



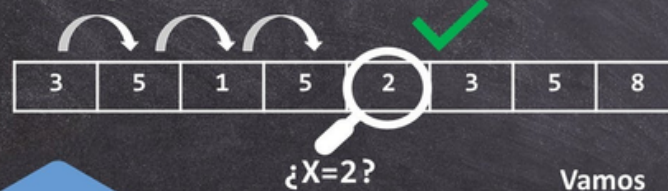
CONTEXTO DE LA TEMÁTICA

Métodos tradicionales

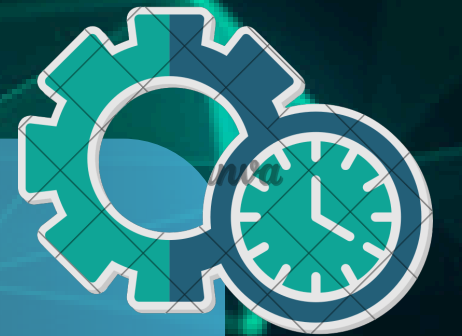


Búsquedas lentas e ineficientes en listas o estructuras secuenciales.

Arreglos: Búsqueda Secuencial



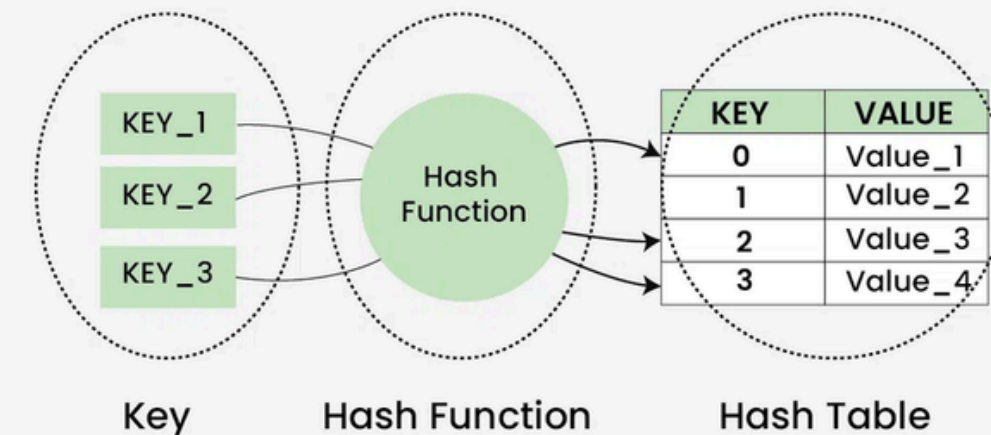
Hashing



Acceso a la información en tiempos muy bajos, cercanos a constante.



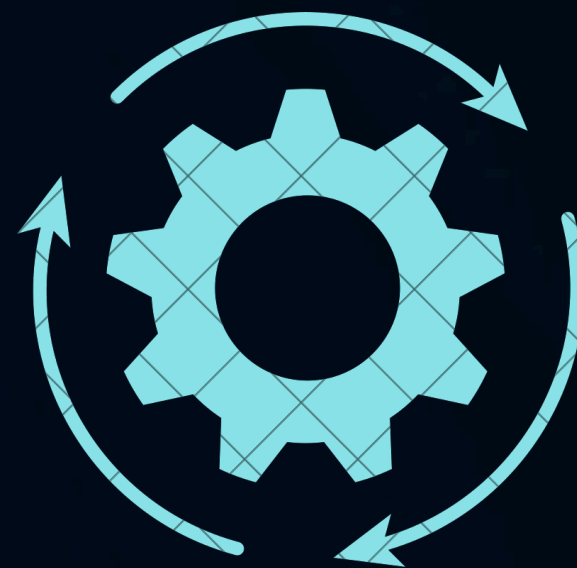
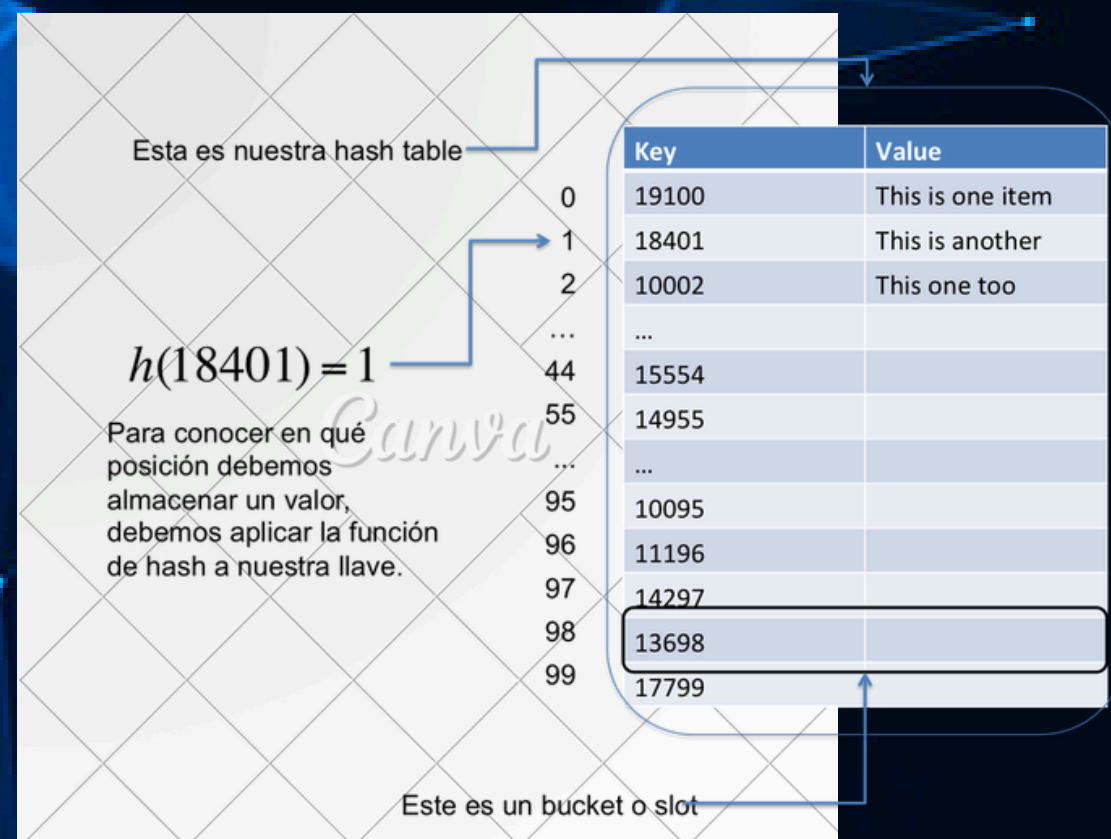
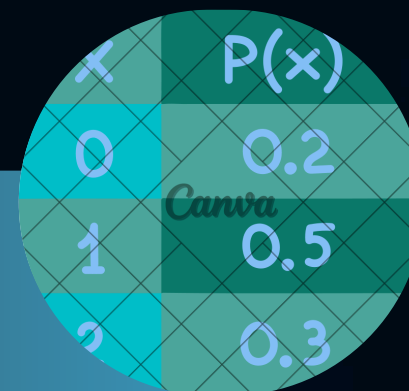
Components of Hashing



IMPLEMENTACIONES DEL HASHING

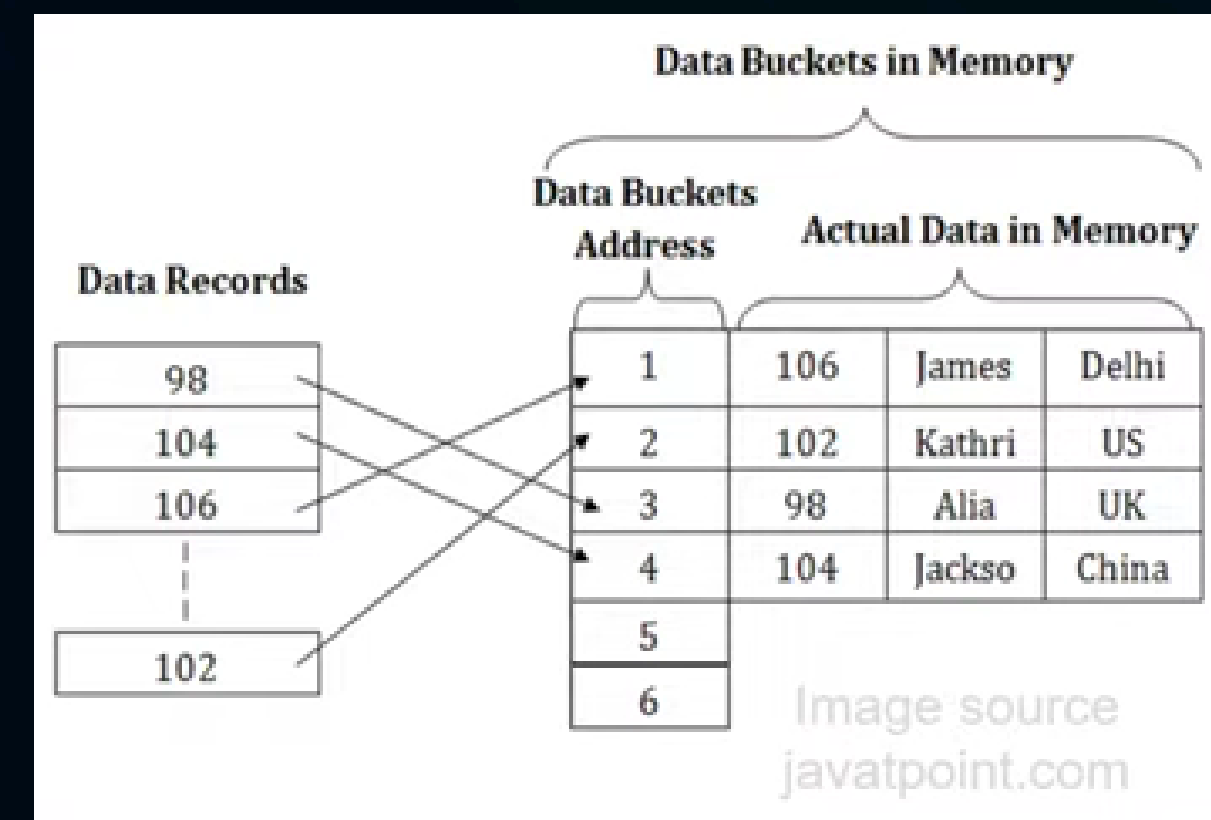
Hash básico

Asignación de claves a posiciones dentro de la tabla y colisiones.



Hashing estático

La tabla tiene un tamaño fijo previamente definido- Puede presentar limitaciones cuando la cantidad de datos crece.

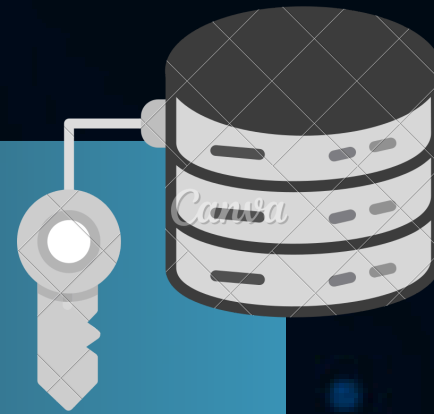




APLICACIONES TÉCNICAS

Bases de datos

Para acceder rápidamente a registros mediante claves.



Estructuras de datos

Como diccionarios o mapas utilizados en programación.

```
'name': 'John'  
'ages': {  
  'Tom': 21  
  'Alice': 19  
}
```



Sistemas de autenticación

Para almacenar contraseñas de forma segura mediante funciones hash.



Sistemas de almacenamiento

Donde el hashing estático se utiliza para organizar archivos en posiciones fijas dentro de la memoria o disco.





UNIVERSIDAD
DE ANTIOQUIA

HASH BASICO



HASH BASICO

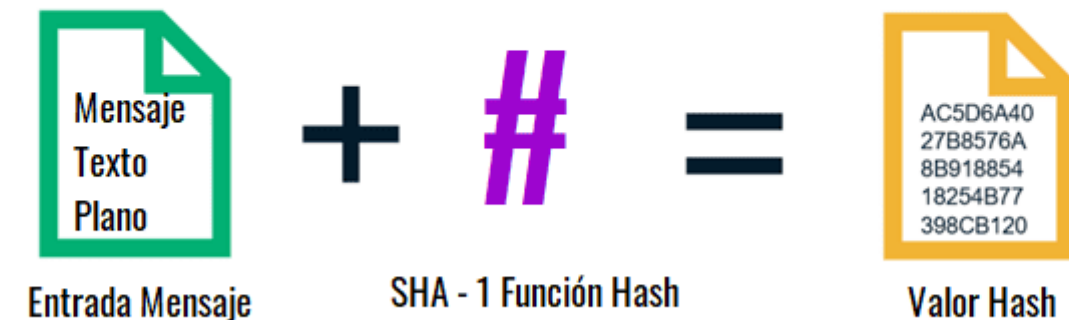
Defincion:

Un hash es una función que toma un dato de entrada (como un número, texto o archivo) y lo transforma en un valor de tamaño fijo, llamado código hash o valor hash.

Funcion Hash:

Es el algoritmo o procedimiento general que define cómo se transforma una entrada en un valor hash de tamaño fijo, siguiendo ciertas propiedades como determinismo, rapidez y distribución uniforme.

EJEMPLO DE FUNCIÓN HASH





¿Cómo funciona?

$$H(K) = K \bmod N$$

K = dato(clave)

M = tamaño de la tabla

$H(K)$ = posición donde se guarda

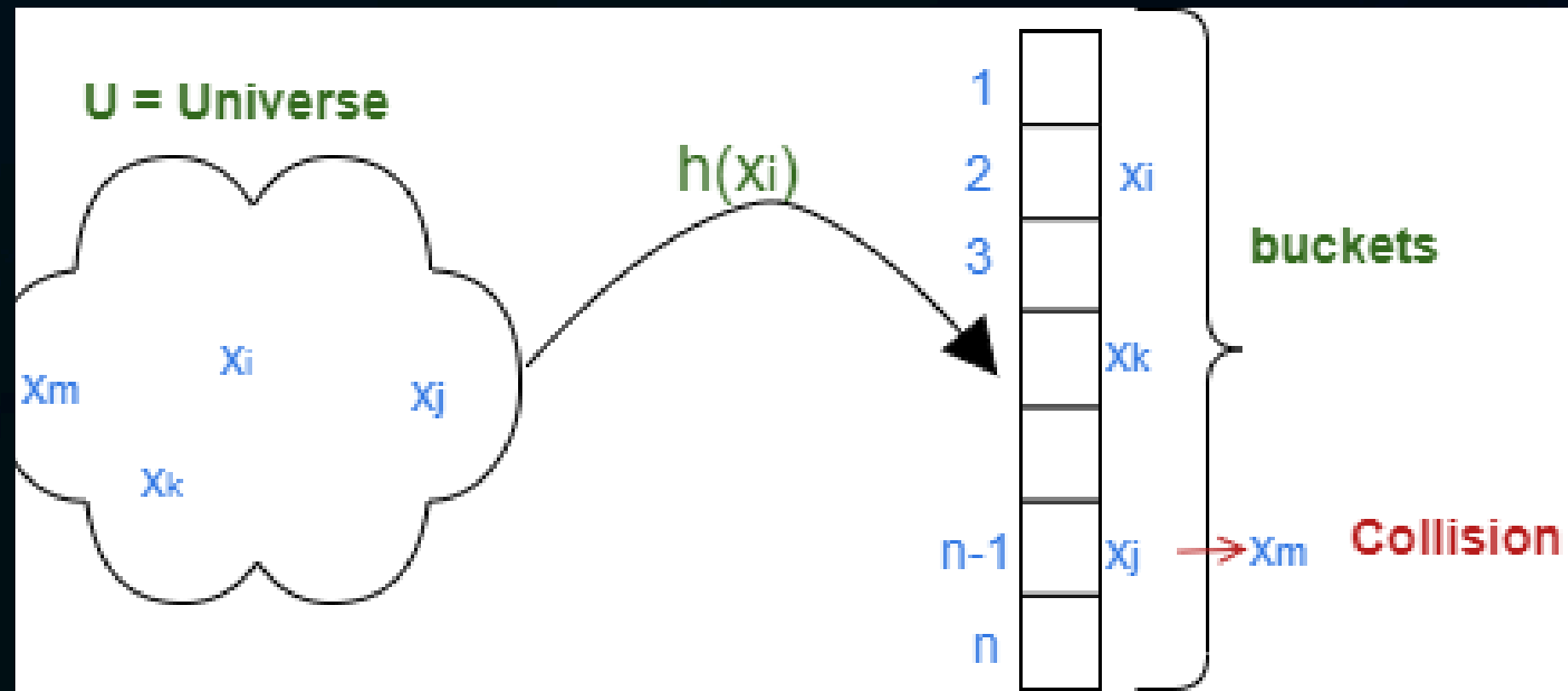
Ejemplo

Si tenemos una tabla de tamaño 10:

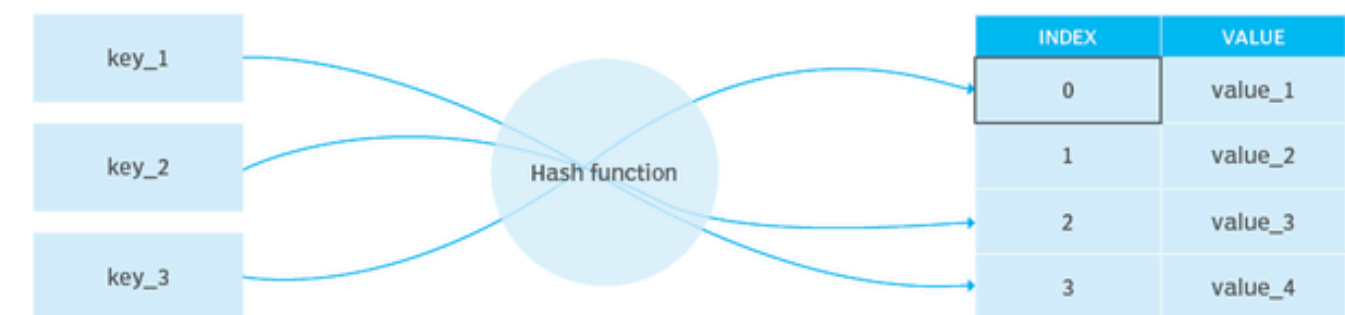
$K = 23$

$H(23) = 23 \bmod 10 = 3$

El dato se guarda en la posición 3



Hash table example





Propiedades

01

Determinista

La misma entrada siempre produce el mismo resultado.

02

Rápida de calcular

Debe generar el valor hash en poco tiempo.

03

Distribución uniforme

Debe repartir los datos de forma equilibrada en la tabla.

04

Pocas colisiones

Debe reducir al máximo que dos claves caigan en la misma posición.

05

Eficiente

Debe mejorar la búsqueda, inserción y recuperación de datos.



UNIVERSIDAD
DE ANTIOQUIA

TÉCNICAS DEL HASH Y MANEJO DE COLISIONES



Las técnicas de Hash



Hashing

Las técnicas de hashing permiten transformar una clave (K) en una posición dentro de una tabla de tamaño (N).

Fórmula general:

$H(K)$ = posición en la tabla

01

Permite acceso rápido a los datos

02

Evita recorrer toda la estructura



Método de división

Fórmula

$$H(K) = \text{mod } N$$

Ejemplo

$$H(20) = 20 \text{ mod } 10 = \underline{0}$$



0

1

2

3

4

5

- Es rápido y fácil de implementar

- Genera colisiones si hay patrones



Método del cuadrado medio

K = 56



Ejemplo

$$56^2 = 3\underline{1}36$$

- Elevar la clave al cuadrado

- Tomar dígitos centrales



Método de plegado

Clave = 12345678

Ejemplo

1234 5678

1234 + 5678 = 6912

- Útil para claves largas

- Puede perder información



Método de multiplicación

Fórmula

$$H(K) = [N (K \cdot A \bmod 1)]$$

- Mejor distribución

- Más complejo

Ejemplo

Si $K=50$, $A=0.618$ y $N=10$

$$50 \cdot 0.618 = 30.950$$

$$0.9 \cdot 10 = 9$$





Manejo de colisiones





UNIVERSIDAD
DE ANTIOQUIA

Direccionamiento abierto



Prueba lineal

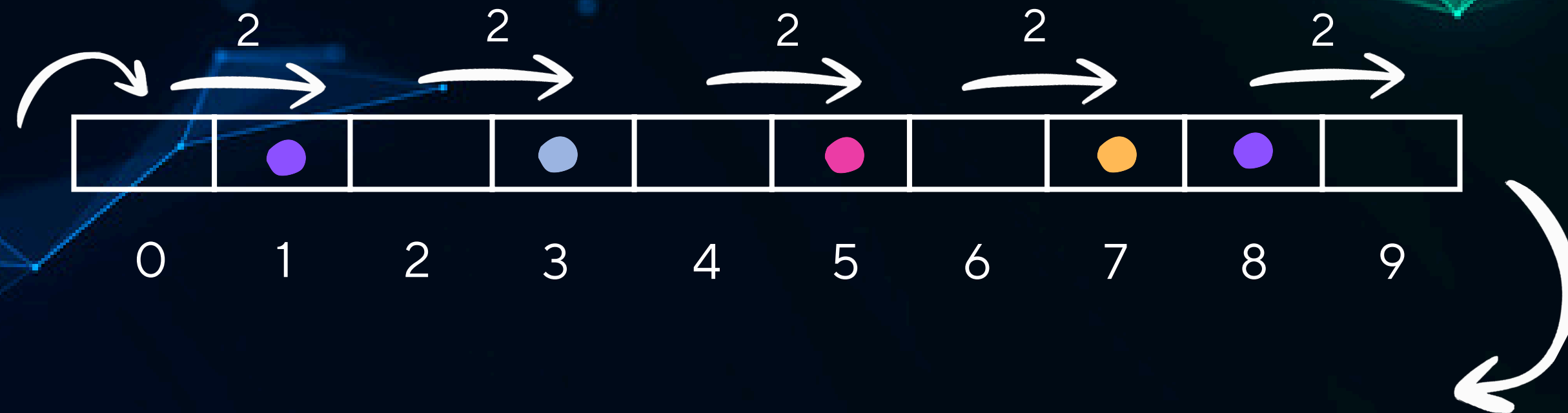
Recorre la tabla secuencialmente para encontrar espacio.





Prueba cuadrática

La prueba cuadrática usa saltos para mejorar la distribución de datos.

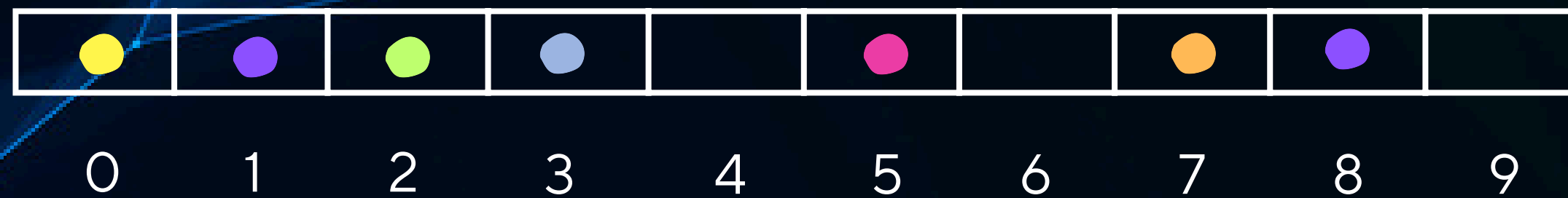




Doble Hashing

El doble hashing usa otra función para ubicar mejor los datos y dispersarlos uniformemente

$$H(K) = \text{mod } N$$





Encadenamiento



Encadenamiento

El encadenamiento almacena varios elementos en una misma posición.





Arreglos anidados

Cada posición contiene otro arreglo para almacenar las colisiones

0	1	2	3	4	5	6	7	8	9
									
									
									



Ventajas y desventajas



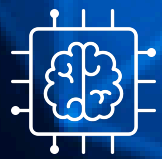
Ventajas



Rapidez en el acceso a la información



Seguridad



Almacenamiento compacto

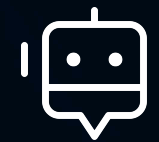


Autenticación e integridad de datos

Desventajas



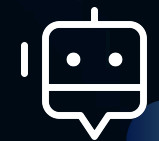
Riesgo de colisiones



No reversible



No adecuado para ordenar datos



Sobrecarga de espacio



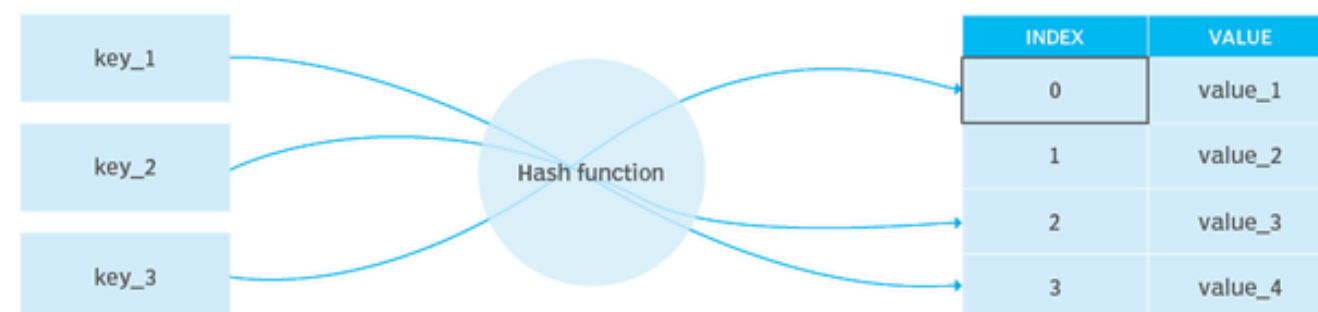
UNIVERSIDAD
DE ANTIOQUIA

APLICACIONES

01 Bases de datos

- Permite ubicar datos de forma directa sin recorrer toda la tabla
- Transforma claves en posiciones específicas (buckets)
- Optimiza búsquedas por igualdad
- Mejora el rendimiento del sistema

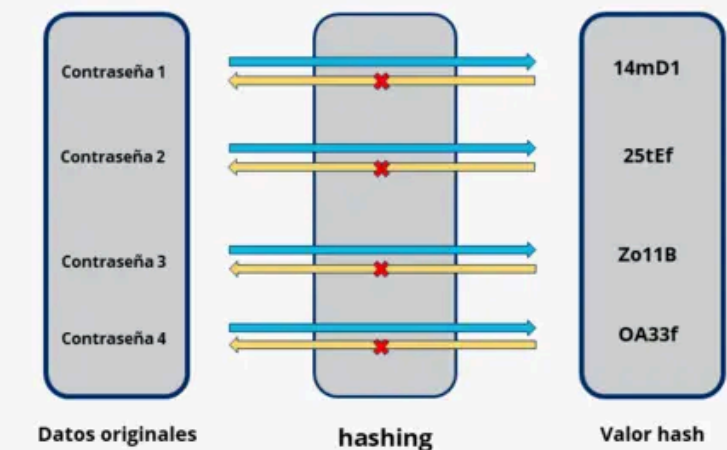
Hash table example



02 Almacenamiento de contraseñas

- No guarda contraseñas en texto plano
- Almacena valores hash para mayor seguridad
- Protege la información ante accesos no autorizados

Funcionamiento del hashing



03 Diccionarios y mapas

- Almacena datos en pares clave-valor
- Permite acceso casi inmediato a la información
- Muy usado en programación (ej: HashMap)

05 Integridad y verificación de datos

- Genera una “huella digital” de la información
- Permite verificar que los datos no han sido modificados
- Se usa en transferencias y almacenamiento de archivos

04 Sistema de almacenamiento

- Organiza grandes volúmenes de datos
- Asigna posiciones específicas a cada elemento
- Facilita recuperación rápida y ordenada





UNIVERSIDAD
DE ANTIOQUIA

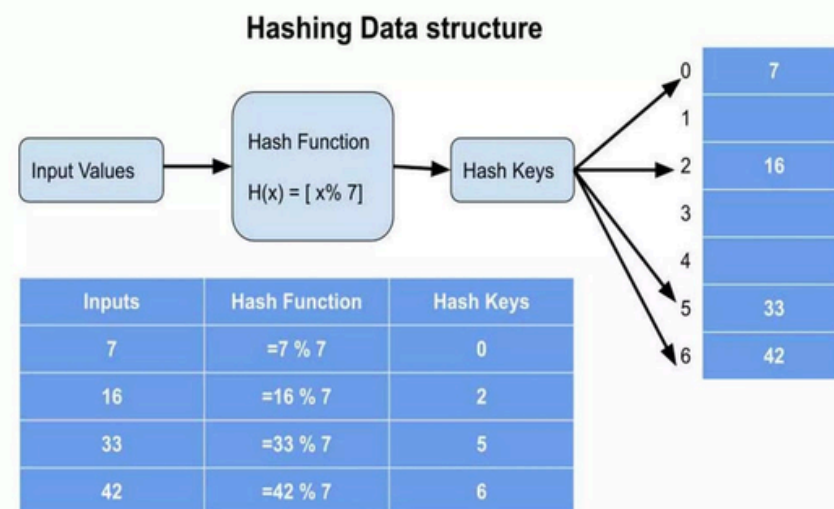
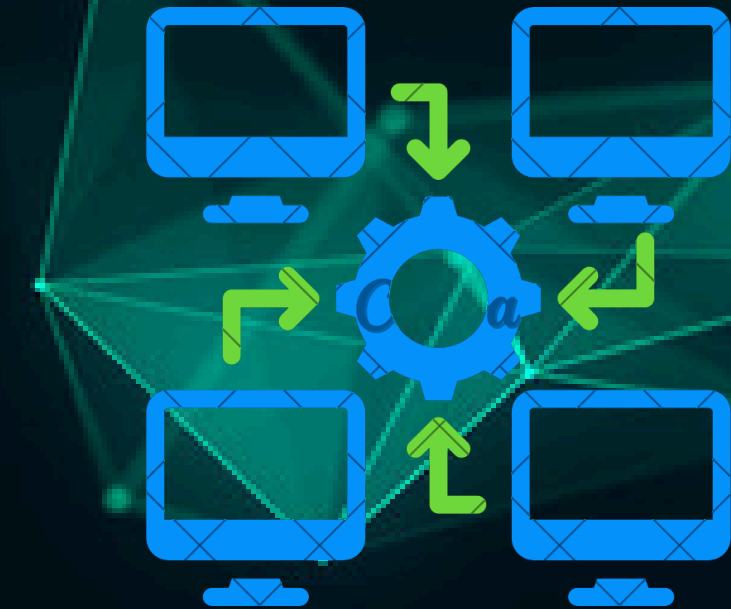
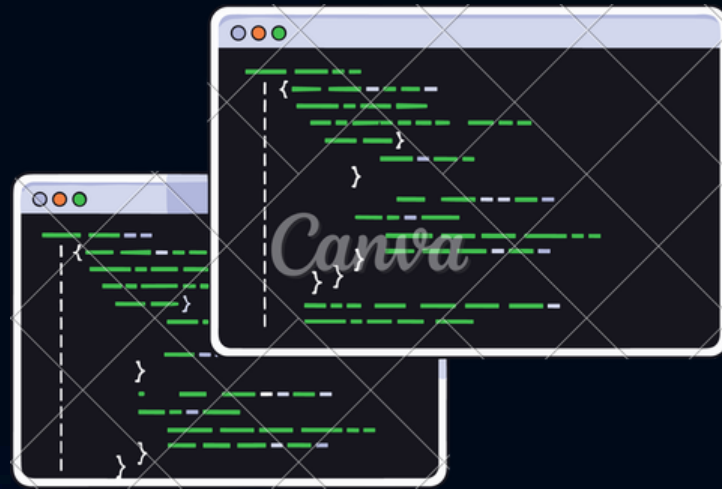
HASHING ESTÁTICO



HASH ESTÁTICO

Definición:

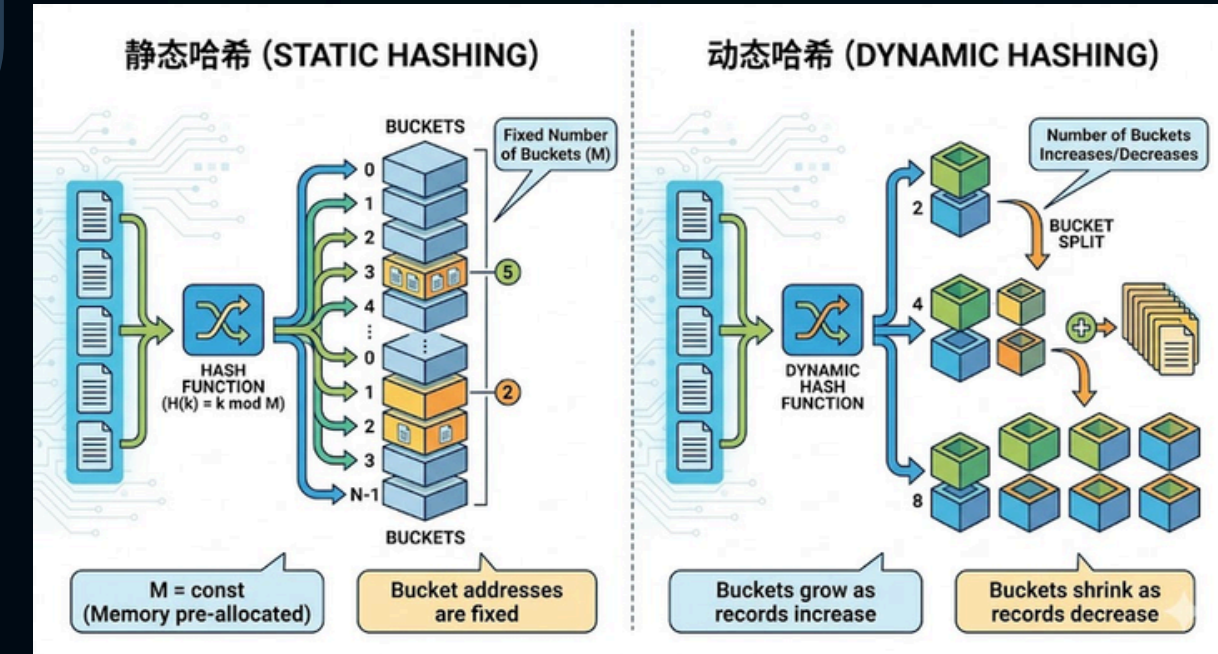
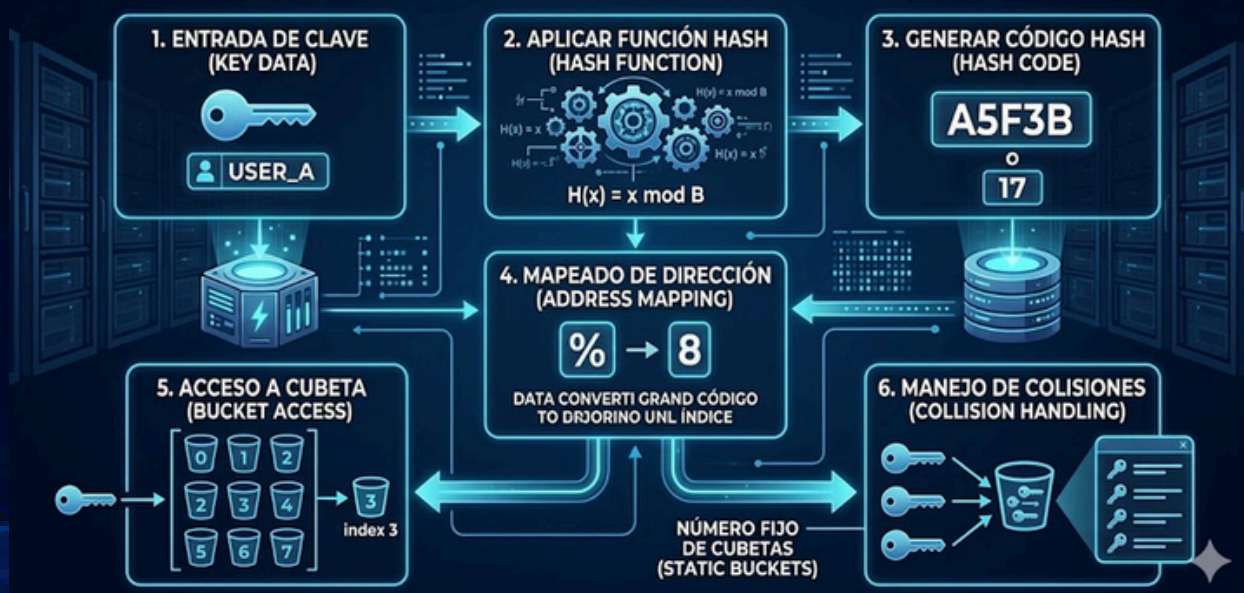
El hashing estático es una técnica de almacenamiento en la que los datos se organizan dentro de una tabla de tamaño fijo, utilizando una función hash que transforma una clave en una posición específica.



Funcionamiento básico

1. Se define una tabla con tamaño fijo (m posiciones)
2. Cada dato tiene una clave
3. Se aplica una función hash:
 $h(\text{clave}) \rightarrow \text{posición}$
4. El dato se almacena en esa posición

PASOS DE FUNCIONAMIENTO BÁSICO DEL HASHING ESTÁTICO

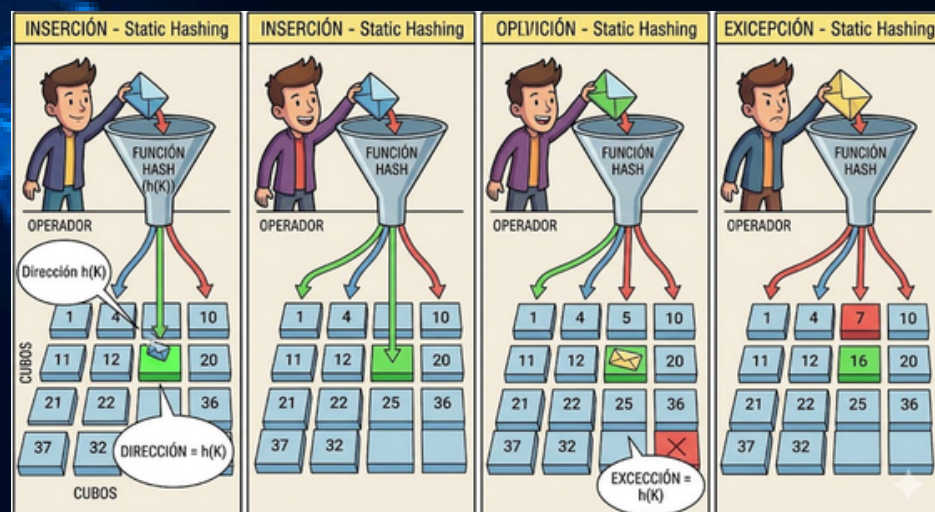


Operaciones del hash estático

01

Inserción

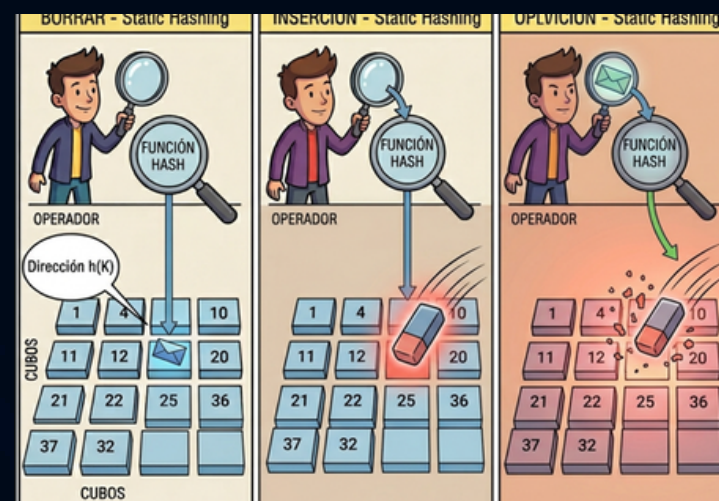
Se aplica la función hash $h(k)$ para calcular la dirección del cubo donde se almacenará el registro.



03

Borrar

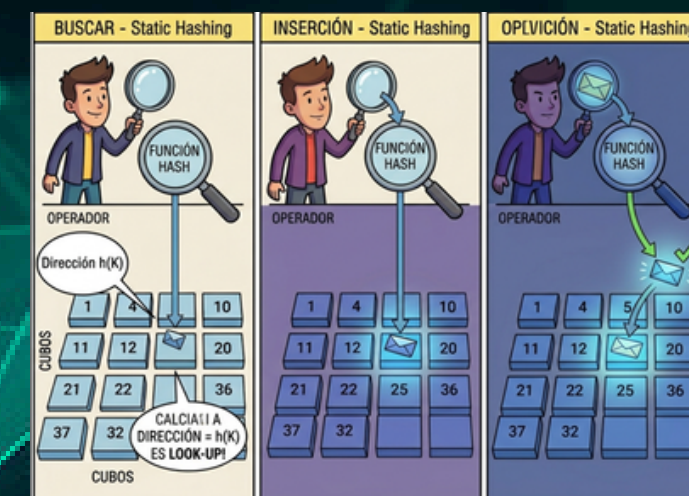
Es posible eliminar los registros de esa dirección en la memoria.



02

Buscar

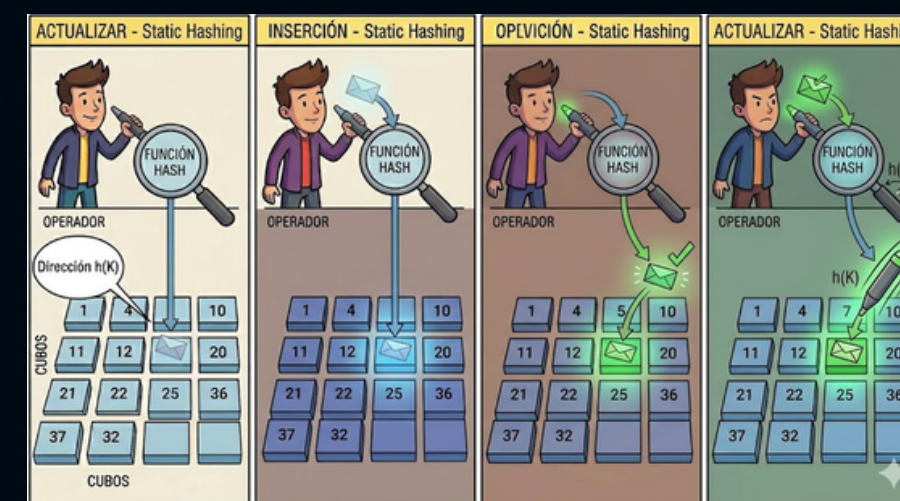
La misma función hash de inserción ayuda a obtener la dirección del depósito donde se almacenan los datos.



04

Actualizar

Después de buscar el registro utilizando una función hash, es posible actualizar ese registro.



Propiedades importantes

01

Tamaño fijo

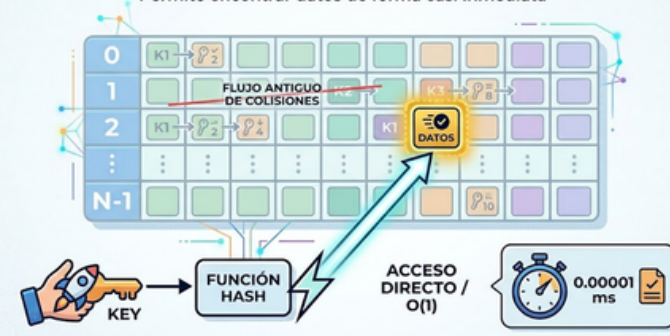
La tabla tiene un número de posiciones definido desde el inicio y no cambia.



TAMAÑO FIJO (ESTÁTICA): El número de posiciones NO CAMBIA



ACCESO RÁPIDO (ESTÁTICO): Búsqueda en $O(1)$
Permite encontrar datos de forma casi inmediata



02

Acceso rápido

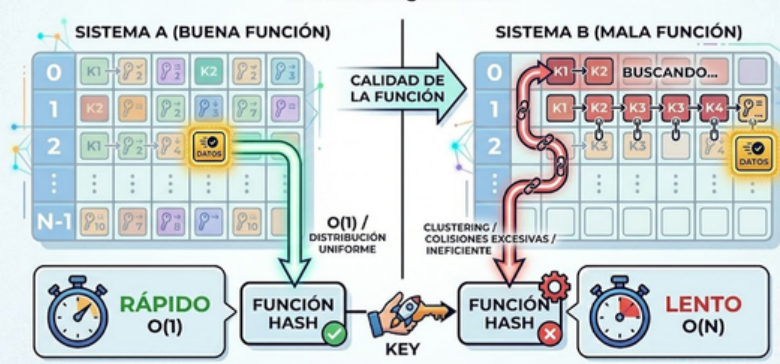
Permite encontrar datos casi de forma inmediata.

03

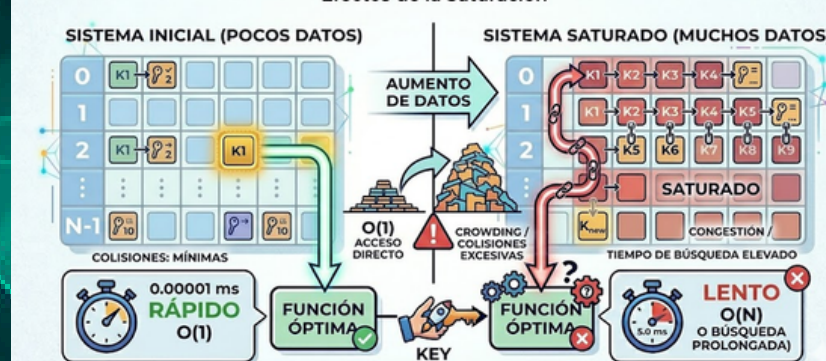
Dependencia de la función hash

La eficiencia depende de qué tan buena sea la función hash.

DEPENDENCIA DE LA FUNCIÓN HASH (ESTÁTICA):
Eficiencia según Calidad



SENSIBILIDAD AL CRECIMIENTO DE DATOS (ESTÁTICA):
Efectos de la Saturación



04

Sensibilidad al crecimiento de datos

A medida que aumentan los datos, aumentan las colisiones.



Ejemplo

Supongamos:

Tamaño de tabla: 7

Estructura con 5 cubos (0 a 4)

Insertión

$k = 12$

$h(12) = 12 \bmod 5 = 2$

Se guarda en el cubo 2

Cubo 2 $\rightarrow$ (12, "Juan")



Índice	Contenido
0	-----
1	-----
2	(12, "Juan")
3	-----
4	-----



Ejemplo

Supongamos:

Tamaño de tabla: 7

Estructura con 5 cubos (0 a 4)

Búsqueda

$k = 12$

$h(12) = 2$

Vamos directamente al cubo 2 y
encontramos el registro

Resultado: (12, "Juan")

Índice	Contenido
0	-----
1	-----
2	(12, "Juan")
3	-----
4	-----



Ejemplo

Supongamos:

Tamaño de tabla: 7

Estructura con 5 cubos (0 a 4)

Borrar

$k = 12$

$h(12) = 2$

Se ubica en el cubo 2 y se elimina

Cubo 2 → vacío

Índice	Contenido
0	-----
1	-----
2	-----
3	-----
4	-----



Ejemplo

Supongamos:

Tamaño de tabla: 7

Estructura con 5 cubos (0 a 4)

Actualizar

Insertamos primero: (7, "Carlos")

$$h(7) = 7 \bmod 5 = 2$$

Ahora queremos actualizar el nombre:

Buscamos: $h(7) = 2$

Cubo 2 → (7, "Carlos")

Cubo 2 → vacío

Se encuentra el registro
y se modifica. Resultado:

Cubo 2 → (7, "Carlos Pérez")



Antes de actualizar

Índice	Contenido
0	-----
1	-----
2	(7, "Carlos")
3	-----
4	-----



Después de actualizar

Índice	Contenido
0	-----
1	-----
2	(7, "Carlos Pérez")
3	-----
4	-----



Ejemplo integrado

Tabla de tamaño fijo (hashing estático): $m = 7$

Función hash (hash básico): $h(k) = k \bmod 7$

Valores a insertar: 10, 22, 31, 4, 15, 28, 17

Aplicación del hash básico

Cálculo de posiciones:

$$h(10) = 10 \bmod 7 = 3$$

$$h(22) = 22 \bmod 7 = 1$$

$$h(31) = 31 \bmod 7 = 3$$

$$h(4) = 4 \bmod 7 = 4$$

$$h(15) = 15 \bmod 7 = 1$$

$$h(28) = 28 \bmod 7 = 0$$

$$h(17) = 17 \bmod 7 = 3$$



Clave	$h(k) = k \bmod 7$
10	3
22	1
31	3 
4	4
15	1 
28	0
17	3 

Aquí ya vemos varias colisiones:

- 31 con 10
- 15 con 22
- 17 con 10 y 31

Resolución con direccionamiento abierto (lineal)

¿Qué hacer?

Buscar la siguiente posición libre.

31 → colisión en 3 → se movió a 4

4 → colisión en 4 → se movió a 5

15 → colisión en 1 → se movió a 2

17 → colisión en 3 → 4 → 5 → se guardó en 6



Índice	Valor
0	28
1	22
2	15
3	10
4	31
5	4
6	17

[Código para visualizar la
implementación del ejemplo](#)



Resolución con encadenamiento

¿Qué hacer?

Las colisiones no se evitan. Entonces se guardan en listas en la misma posición.

0 → 28

1 → 22 → 15

2 → ---

3 → 10 → 31 → 17

4 → 4

5 → ---

6 → ---



Índice	Valor
0	28
1	22 → 15
2	---
3	10 → 31 → 17
4	4
5	---
6	---

Código para visualizar la
implementación del ejemplo



Hash estático

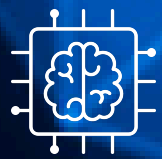
Ventajas



Rendimiento muy alto



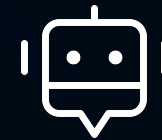
Complejidad cercana a $O(1)$



Utiliza el valor de la clave primaria como entrada de la función hash.



No se adapta bien al crecimiento de los datos



Desbordamiento de buckets

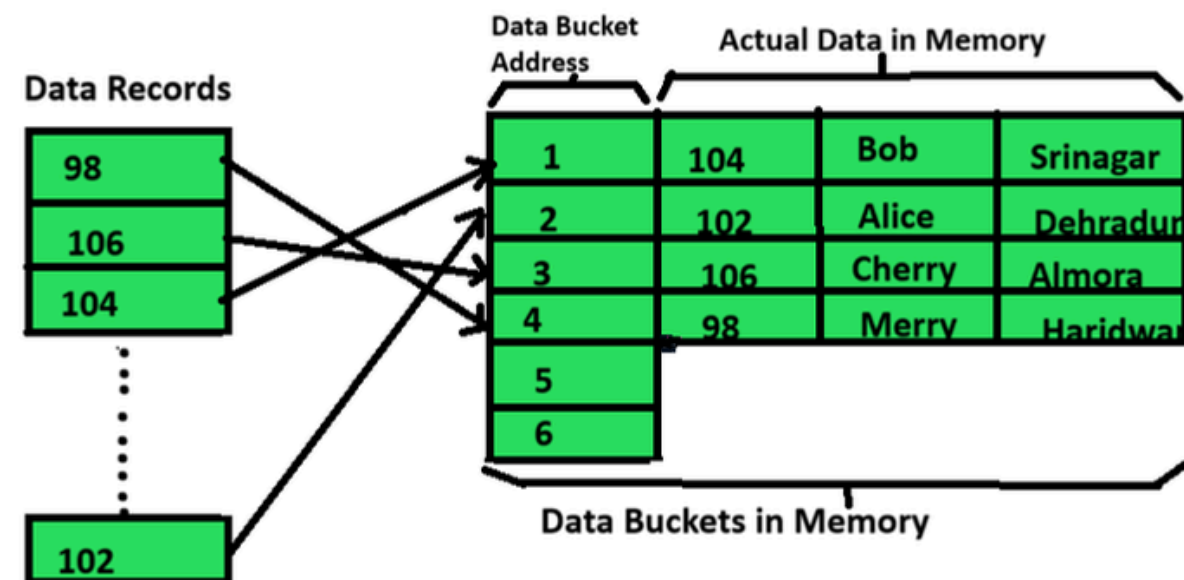


UNIVERSIDAD
DE ANTIOQUIA

APLICACIONES

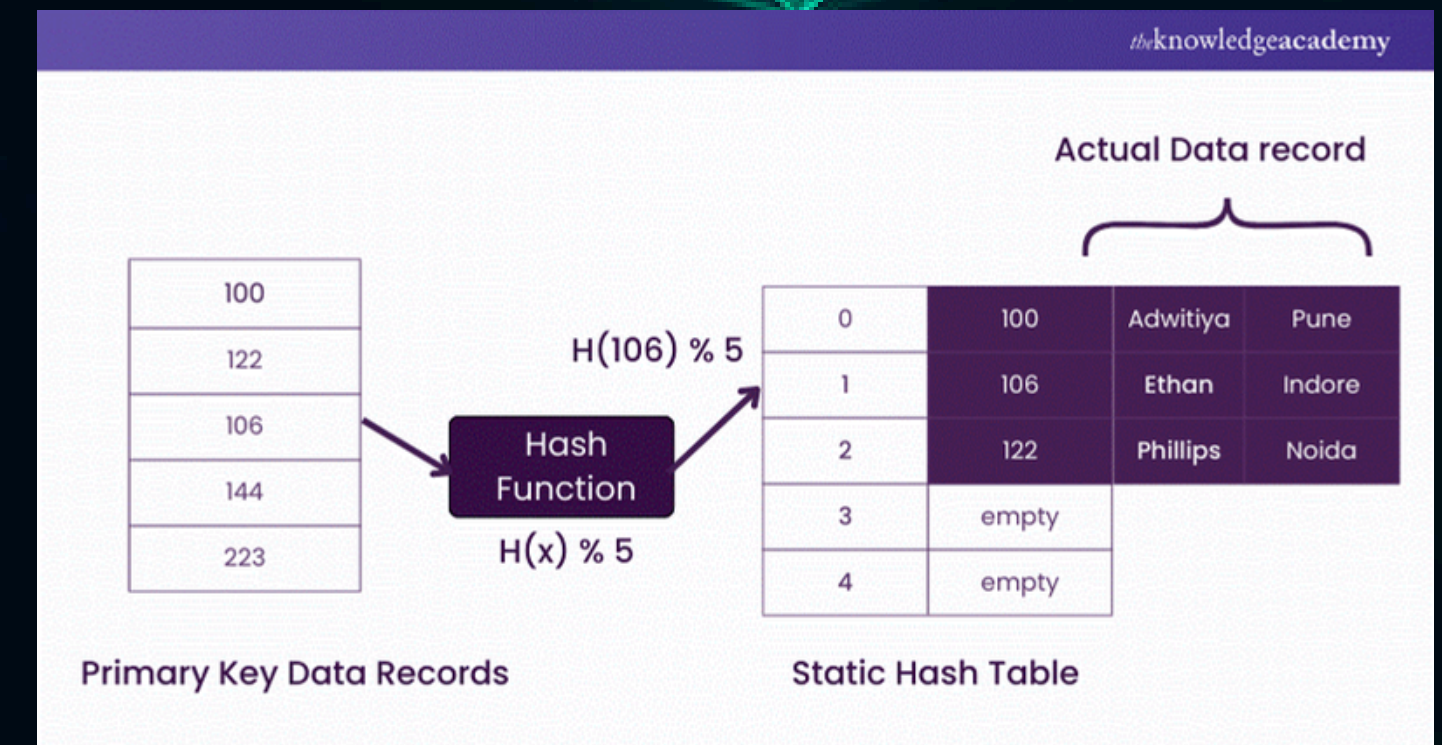
01 Bases de datos con tamaño definido

- Se usa cuando la cantidad de datos es estable
- Maneja un número fijo de buckets
- Permite búsquedas rápidas por igualdad
- Acceso directo sin recorrer toda la base de datos



02 Sistemas de almacenamiento de archivos

- Organiza y localiza archivos de forma eficiente
- Asocia cada dato a una posición mediante hashing
- Facilita la recuperación rápida de información
- Puede presentar desbordamientos si los datos crecen





UNIVERSIDAD
DE ANTIOQUIA

CONCLUSIONES



El hashing permite acceder a los datos de forma rápida y eficiente.

Es clave para la seguridad y el rendimiento de los sistemas.



Requiere una buena función hash y manejo de colisiones para ser confiable.

**MUCHAS
GRACIAS**



**UNIVERSIDAD
DE ANTIOQUIA**

1 8 0 3